52. Evaluating AI Systems in Go

52. Evaluating AI Systems in Go

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `52-eval-harnesses`

Learning objectives

- Build offline eval datasets
- Measure latency/cost/quality
- Automate regression evals in CI
- Separate unit tests from model evals

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Evals make AI systems engineering, not vibes. Harnesses score outputs so changes are measurable.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Golden sets: input → expected properties.
- Exact match, contains, regex, LLM-as-judge (careful).
- Track regressions in CI.
- Separate retrieval vs generation metrics for RAG.
- Seed randomness for determinism when possible.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-evaluating && cd /tmp/gw-evaluating
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "strings"
)

func ScoreContains(got, want string) float64 {
    if strings.Contains(got, want) {
        return 1
    }
    return 0
}

func main() { fmt.Println(ScoreContains("hello world", "world")) }
```

Run it:

```
go run .
```

Expected output:

```
1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Score a tiny golden set average.
2. Add a case that fails; see average drop.
3. Store results as JSON for a report.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

What to measure

- **Latency**: p50/p95 end-to-end
- **Cost**: tokens / $ per request
- **Quality**: exact match, rubrics, LLM-as-judge (carefully), retrieval recall@k

Harness

```
for _, tc := range dataset {
    started := time.Now()
    out, err := system.Run(ctx, tc.In)
    // record metrics + score(out, tc.Want)
}
```

Gate releases on eval thresholds. Keep golden sets versioned.

Unit-test pure functions (chunking, cosine); eval the full system less frequently.

Failure modes to watch while coding

- Only vibe-checking prompts
- Flaky online evals without seeds
- Optimizing cost while quality collapses

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- pytest + eval frameworks — same discipline.
- Notebooks are not CI.
- Human eval samples still matter.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Optimize for the metric you measure — choose metrics that match user value.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Evaluating AI Systems in Go” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Only vibe-checking prompts
- Flaky online evals without seeds
- Optimizing cost while quality collapses

Exercises

Work in `exercises/52-eval-harnesses/`. Read that folder’s `README.md` before you code.

What you will build

Implement scoring helpers for the eval harness exercise.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/52-eval-harnesses/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Name three metric classes
2. Why golden sets?
3. What belongs in CI vs nightly?

Further reading

- https://go.dev/blog/survey2024-overview